

### Практическая работа №3. Елкин Дмитрий.

Код который я написал:

```
x_val = 42

pi_num = 3.14159

msg = "Hello, Python!"

flag = True

arr = [1, 2, 3]

tpl = (4, 5, 6)

dct = {"name": "Alice", "age": 30}

st = {7, 8, 9}

empty_val = None


print('Тип переменной x_val:', type(x_val))
print('Тип переменной pi_num:', type(pi_num))
print('Тип переменной msg:', type(msg))
print('Тип переменной flag:', type(flag))
print('Тип переменной arr:', type(arr))
print('Тип переменной tpl:', type(tpl))
print('Тип переменной dct:', type(dct))
print('Тип переменной st:', type(st))
print('Тип переменной empty_val:', type(empty_val))
```

Компиляция кода:

```
Тип переменной x_val: <class 'int'>
Тип переменной pi_num: <class 'float'>
Тип переменной msg: <class 'str'>
Тип переменной flag: <class 'bool'>
Тип переменной arr: <class 'list'>
Тип переменной tpl: <class 'tuple'>
Тип переменной dct: <class 'dict'>
Тип переменной st: <class 'set'>
```

```
Тип переменной empty_val: <class 'NoneType'>
```

## Задание 1

1) Код который я написал с комментарием:

```
my_items = [1, 2, 3]

print('Стартовый список:', my_items)

my_items[0] = 100

print('Обновленный список:', my_items)

# Изменил нулевой элемент списка
```

Компиляция кода:

```
Стартовый список: [1, 2, 3]

Обновленный список: [100, 2, 3]
```

2) Код который я написал с комментарием:

```
my_tuple_val = (1, 2, 3)

print('Стартовый кортеж:', my_tuple_val)

# my_tuple_val[0] = 100

print('Попытка изменить кортеж завершилась ошибкой', my_tuple_val)

# Пытался изменить кортеж, но из-за его неизменяемости ничего не вышло
```

Компиляция кода:

```
TypeError: 'tuple' object does not support item assignment

Стартовый кортеж: (1, 2, 3)
```

3) Код который я написал с комментарием:

```
text_val = 'cat'

print('Изначальная строка:', text_val)

# text_val[0] = 'b'

print('Измененная строка:', text_val)

# Изменить символ в строке нельзя, так как строки неизменяемы
```

Компиляция кода:

```
TypeError: 'str' object does not support item assignment
```

Изначальная строка: cat

## Задание 2

Изначальный код без try-except:

```
num_a = input()
num_b = input()
num_b = int(num_b)
num_a = int(num_a)
print(num_a + num_b)
```

Компиляция с числами 3 и 5:

8

Компиляция с числами 3 и 4.5:

```
ValueError: invalid literal for int() with base 10: '4.5'
```

Код с try-except:

```
try:
    num_a = input()
    num_b = input()
    num_b = int(num_b)
    num_a = int(num_a)
    print(num_a + num_b)
except:
    print("Ошибка: введены неверные данные.")
```

## Задание 3

- Для "Хранение текущего количества студентов в группе." я выберу переменную **int()**, так как она предназначена для целых чисел.
- Для "Хранение температуры в градусах Цельсия (например, 23.5)" я выберу переменную **float()**, так как она хранит вещественные числа.
- Для "Хранение логина пользователя (например, "ivanov\_ivan")" я выберу переменную **str()**, так как текст сохраняется в виде строки.
- Для "Хранение информации о том, включено ли уведомление (да/нет)" я выберу переменную **bool()**, так как есть только два варианта: True или False.

- Для "Хранение списка любимых фильмов" я выберу переменную **list()**, ведь элементов может быть много и коллекцию удобно дополнять.
- Для "Хранение координат точки в 2D пространстве (x, y), которые не должны изменяться" я выберу переменную **tuple()**, так как кортеж защищен от изменений.
- Для "Хранение уникальных ID товаров в корзине покупок" я выберу переменную **set()**, так как она автоматически убирает дубли.
- Для "Хранение соответствия между названием страны и её столицей" я выберу переменную **dict()**, так как словарь связывает пары ключ-значение.

## Задание 4

Код который я написал:

```
fruits_list = ['яблоко', 'банан', 'груша']

fruits_list.append('апельсин')

print('Список фруктов:', fruits_list)

unique_fruits = set(fruits_list)

print('Множество фруктов:', unique_fruits)
```

Компиляция кода:

```
Список фруктов: ['яблоко', 'банан', 'груша', 'апельсин']

Множество фруктов: {'яблоко', 'груша', 'банан', 'апельсин'}
```

set() и list() различаются тем, что set() не допускает дубликатов и не сохраняет исходный порядок элементов, в отличие от list().

## Задание 5

```
user_name = input('Ваше имя: ')

user_age_str = input('Ваш возраст: ')

hobbies_str = input('Хобби через запятую: ')

user_age = int(user_age_str)

hobbies_list = [item.strip() for item in hobbies_str.split(",")]

profile = {

    'name': user_name,

    'age': user_age,

    'hobbies': hobbies_list

}

print('=' * 30)
```

```
print('ПРОФИЛЬ ПОЛЬЗОВАТЕЛЯ')

print('=' * 30)

print('Имя:', profile['name'])

print('Возраст:', profile['age'])

print('Хобби:', profile['hobbies'])

print('=' * 30)
```

## Контрольные вопросы

1. **Какие типы данных являются простыми (скалярными)?** Это типы, хранящие одно значение: целые числа `int()`, вещественные `float()`, логические `bool()` и строки `str()`.
2. **В чём разница между `list()` и `tuple()`?** Главное различие — изменяемость. `list()` можно модифицировать, а `tuple()` является неизменяемым.
3. **Почему важно понимать, является ли тип данных изменяемым или нет?** Это критично для предотвращения багов и корректного использования элементов в словарях и множествах.
4. **Зачем нужно преобразование типов? В каких случаях оно обязательно?** Оно нужно для совместимости данных, например, при обработке ввода `input()`, который всегда возвращает строку.
5. **Какие ошибки могут возникнуть при преобразовании, и как с ними бороться?** Возникают ошибки вида `ValueError`, с которыми борются с помощью конструкции `try-except`.